

Machine Learning LAB

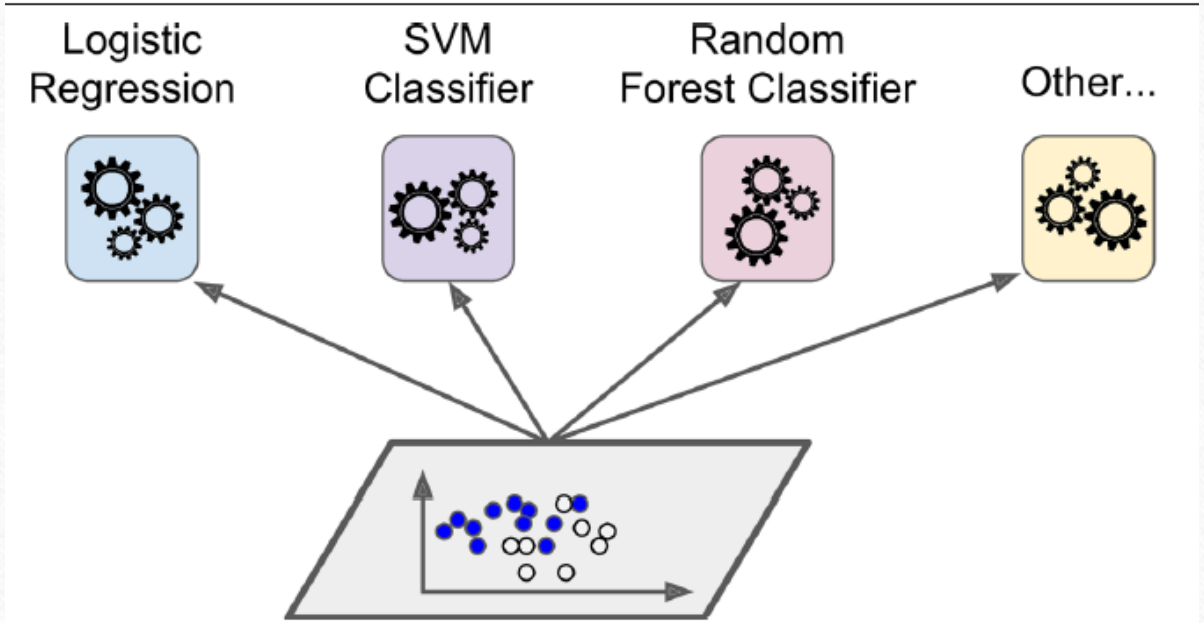
Week 05

Topics to cover

- Voting Classifier
- K-mean clustering

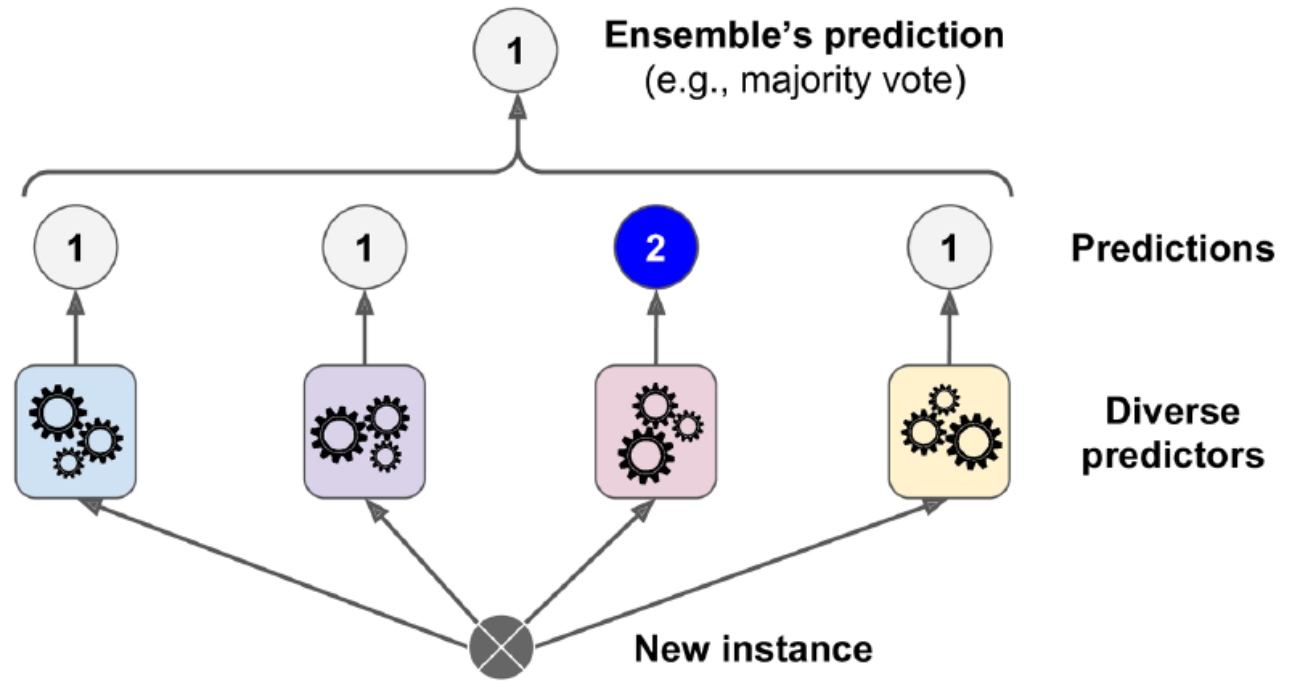
Voting Classifiers

- Suppose you have trained a few classifiers, each one achieving about 80% accuracy. You may have a Logistic Regression classifier, an SVM classifier, a Random Forest classifier, a K-Nearest Neighbors classifier, and perhaps a few more



Voting Classifiers

- A very simple way to create an even better classifier is to aggregate the predictions of each classifier and predict the class that gets the most votes. This majority-vote classifier is called a *hard voting* classifier



Voting Classifiers

- from sklearn.ensemble import RandomForestClassifier
- from sklearn.ensemble import VotingClassifier
- from sklearn.linear_model import LogisticRegression
- from sklearn.svm import SVC
- log_clf = LogisticRegression()
- rnd_clf = RandomForestClassifier()
- svm_clf = SVC()
- voting_clf = VotingClassifier(
- estimators=[('lr', log_clf), ('rf', rnd_clf), ('svc', svm_clf)], voting='hard')
- voting_clf.fit(X_train, y_train)
- print(voting_clf.predict([some_digit]))

Voting Classifiers

- from sklearn.metrics import accuracy_score
- for clf in (log_clf, rnd_clf, svm_clf, voting_clf):
- clf.fit(X_train, y_train)
- y_pred = clf.predict(X_test)
- print(clf.__class__.__name__, accuracy_score(y_test, y_pred))

```
LogisticRegression 0.9195171026156942
RandomForestClassifier 0.9255533199195171
SVC 0.9537223340040242
```

K-Means Clustering

- 1 - Imports
- 2 - Loading the data
- 3 - Exploratory Analysis
- 4 - Preprocessing the data
- 5 - Trainin K-Means Clustering to Find Clusters
- 6 - Evaluating K-Means Clustering

IMPORTS AND DATA LOADING

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt


# Step 1: Load the dataset
data = pd.read_csv("/content/mushroomss.csv")


# Display first few rows
print("Dataset Preview:")
print(data.head())


# Step 2: Check for missing values
print("\nMissing Values in Each Column:")
print(data.isnull().sum())
```

EXPLORATORY DATA ANALYSIS

```
print(f"Dataset shape: {mushroom_data.shape}")
print(f"Target
distribution:\n{mushroom_data['class'].value_counts()}")
```

```
Dataset shape: (8124, 23)
Target distribution:
class
e    4208
p    3916
Name: count, dtype: int64
```

EXPLORATORY DATA ANALYSIS

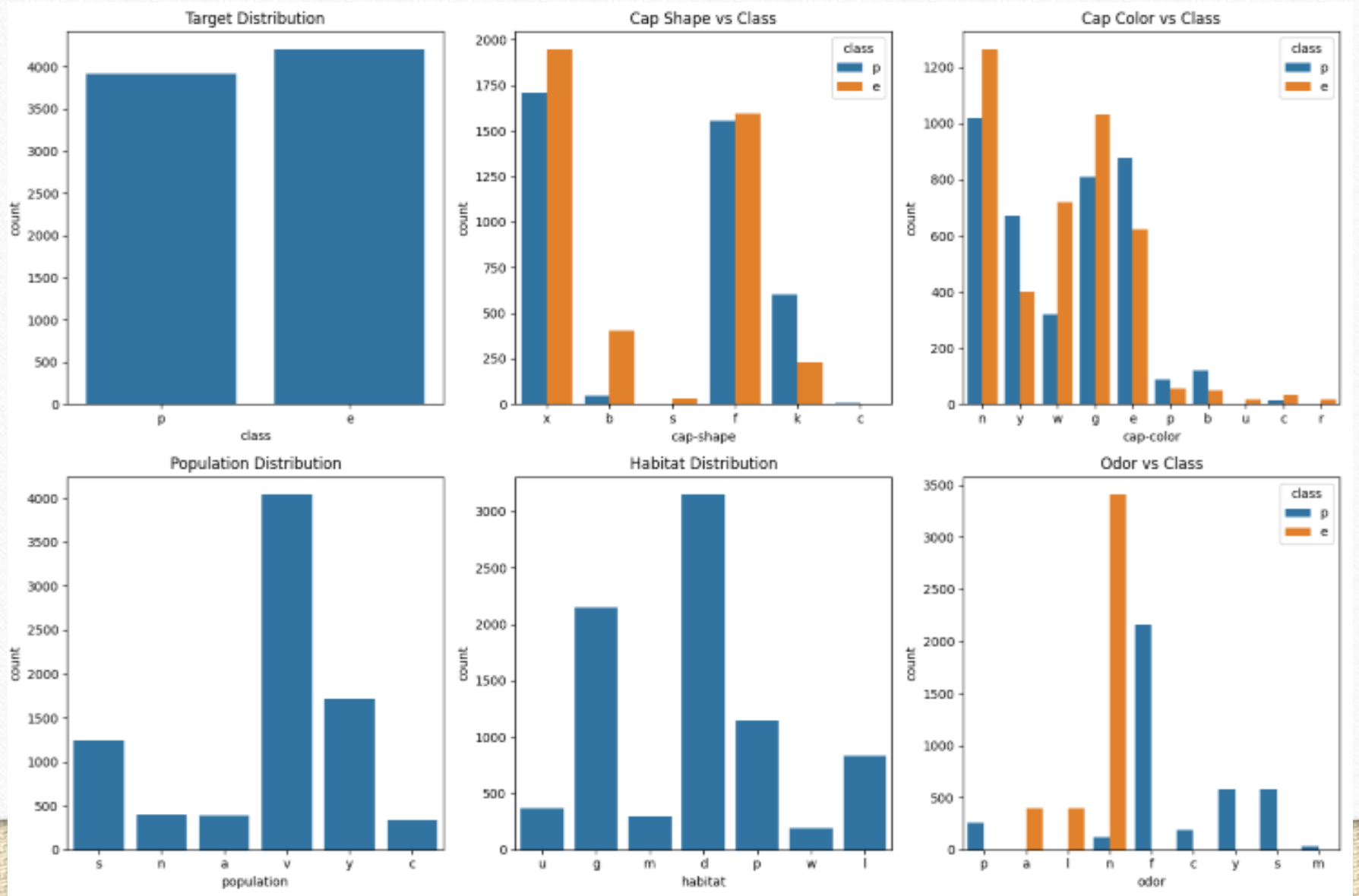
```
plt.figure(figsize=(15, 10))
plt.subplot(2, 3, 1)
sns.countplot(data=mushroom_data, x='class')
plt.title('Target Distribution')

plt.subplot(2, 3, 2)
sns.countplot(data=mushroom_data, x='cap-shape', hue='class')
plt.title('Cap Shape vs Class’)
plt.subplot(2, 3, 3)
sns.countplot(data=mushroom_data, x='cap-color', hue='class')
plt.title('Cap Color vs Class')
```

EXPLORATORY DATA ANALYSIS

```
plt.subplot(2, 3, 4)
sns.countplot(data=mushroom_data, x='population')
plt.title('Population Distribution')
plt.subplot(2, 3, 5)
sns.countplot(data=mushroom_data, x='habitat')
plt.title('Habitat Distribution')
plt.subplot(2, 3, 6)
sns.countplot(data=mushroom_data, x='odor', hue='class')
plt.title('Odor vs Class')
plt.tight_layout()
plt.show()
```

EXPLORATORY DATA ANALYSIS



DATA PREPROCESSING

```
#  Handle missing values
mushroom_df = mushroom_data.dropna()

# Separate features and target
mushroom_features = mushroom_df.drop('class', axis=1)
mushroom_labels = mushroom_df['class']
# Encode categorical features
encoder = OrdinalEncoder()
mushroom_prepared =
    encoder.fit_transform(mushroom_features)
mushroom_prep_df = pd.DataFrame(mushroom_prepared,
columns=mushroom_features.columns)
```

MODEL IMPLEMENTATION

KMEANS CLUSTERING

```
# Create and fit KMeans model
k_clust = KMeans(n_clusters=2, random_state=42)

k_clust.fit(mushroom_prep_df)

# Get results
k_labels = k_clust.labels_
```

MODEL EVALUATION

```
 # Map true labels to numerical values
map_dict = {'e': 0, 'p': 1}
true_labels = mushroom_labels.map(map_dict)


# Calculate accuracy
accuracy = accuracy_score(true_labels, k_labels)
print(f"KMeans Accuracy: {accuracy:.4f}")
```

```
KMeans Accuracy: 0.7090
```

Confusion matrix

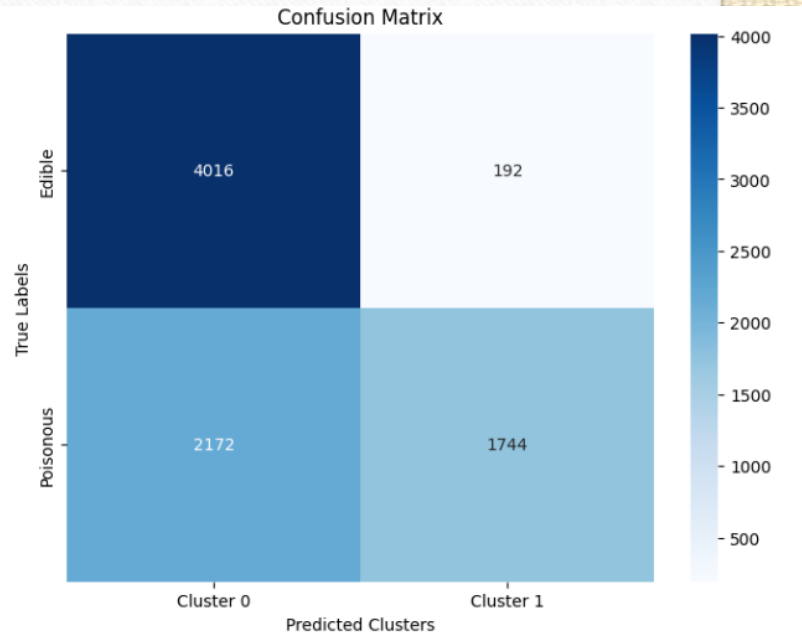
```
plt.figure(figsize=(8, 6))

cm = confusion_matrix(true_labels, k_labels)

sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Cluster 0', 'Cluster 1'],
            yticklabels=['Edible', 'Poisonous'])

plt.title('Confusion Matrix')
plt.xlabel('Predicted Clusters')
plt.ylabel('True Labels')
plt.show()

# Classification report
print("\nClassification Report:")
print(classification_report(true_labels, k_labels,
target_names=['Edible', 'Poisonous']))
```



Classification Report:

	precision	recall	f1-score	support
Edible	0.65	0.95	0.77	4208
Poisonous	0.90	0.45	0.60	3916
accuracy			0.71	8124
macro avg	0.77	0.70	0.68	8124
weighted avg	0.77	0.71	0.69	8124

UNSUPERVISED LEARNING

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt


# Step 1: Load the dataset
data = pd.read_csv("/content/mushroomss.csv")


# Display first few rows
print("Dataset Preview:")
print(data.head())


# Step 2: Check for missing values
print("\nMissing Values in Each Column:")
print(data.isnull().sum())


# Step 3: Encode categorical features (if any)
label_encoders = {}
for column in data.columns:
    if data[column].dtype == 'object':
        le = LabelEncoder()
        data[column] = le.fit_transform(data[column])
        label_encoders[column] = le
```

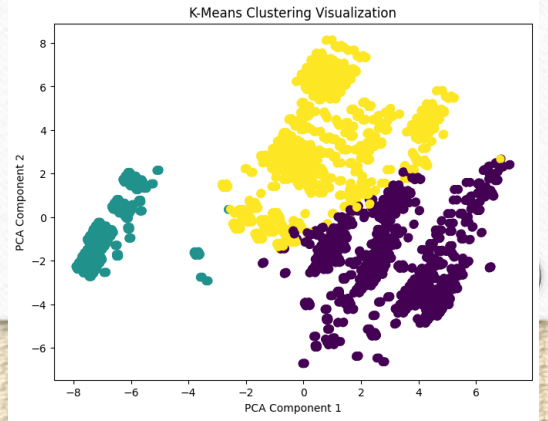
UNSUPERVISED LEARNING

```
# Step 4: Apply K-Means Clustering
kmeans = KMeans(n_clusters=3, random_state=42)
data['Cluster'] = kmeans.fit_predict(data)


# Step 5: Display cluster centers and counts
print("\nCluster Centers:\n", kmeans.cluster_centers_)
print("\nCluster Counts:\n", data['Cluster'].value_counts())


# Step 6: Visualize clusters using PCA (2D)
pca = PCA(n_components=2)
reduced_data = pca.fit_transform(data.drop('Cluster', axis=1))
plt.figure(figsize=(8,6))
plt.scatter(reduced_data[:,0], reduced_data[:,1],
c=data['Cluster'], cmap='viridis', s=50)
plt.title('K-Means Clustering Visualization')
plt.xlabel('PCA Component 1')
plt.ylabel('PCA Component 2')
plt.show()
```

```
Cluster Counts:
 Cluster
0    3535
2    2707
1    1882
```



Activity 1

KNN for Credit Card Fraud Detection

Data Set: https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud?resource=download

Use KNN to detect fraudulent transactions in a highly **imbalanced dataset**, where frauds make up less than 0.2% of all samples.

1. Load the dataset and split it into features X and target y.
2. Apply **SMOTE (Synthetic Minority Oversampling Technique)** or **ADASYN** to handle class imbalance.
3. Scale features using **StandardScaler**.
4. Train a **KNN classifier** with k=5, then tune k using **GridSearchCV** for best results.
5. Compare model performance using:
    1. Accuracy
    2. F1-Score

Activity 2

SVM for Brain Tumor Detection

Data Set: https://www.kaggle.com/datasets/navoneel/brain-mri-images-for-brain-tumor-detection
Load the dataset and preprocess all MRI images:
Convert to grayscale
Resize to 64×64
Flatten each image
Split data into training and testing sets (80:20).
Normalize features using StandardScaler().
Train an **SVM (RBF kernel)** with C=10 and gamma='scale'.
Evaluate performance using accuracy, classification report, and confusion matrix.
Display a few test images with predicted and actual labels.

Activity 3

Dataset:
`Mall_Customers`
Tasks:
1.Load the dataset and display it.
2.Apply **K Mean** & **DBSCAN Algorithm** and plot clusters using *Annual Income* vs *Spending Score*.
3.Scale features using **StandardScaler** and re-run K-Means.
4.Compare both results did scaling change clusters?
5.Use the **Elbow Method** to find the best `k`.
6.Briefly interpret each cluster.
Also:
Add a fake feature `"LuckyNumber"` (random 0–100) and explain its effect on clustering.